Técnicas de computación paralela y plataformas cloud

Investigación y actividad de exposición

ELVY ANTONIO GRULLÓN MOREL

Matrícula: 1222572

Caso práctico: UserFlow

Octubre de 2026

Introducción

La computación paralela busca ejecutar trabajo simultáneamente para reducir tiempo, aumentar capacidad o mantener disponible un sistema. La concurrencia organiza tareas que progresan durante el mismo intervalo; el paralelismo ejecuta trabajo al mismo tiempo en varios núcleos o máquinas. Esta investigación explica técnicas algorítmicas, componentes de infraestructura y herramientas de automatización, y las conecta con la aplicación UserFlow.

Objetivos

- Definir cada concepto con lenguaje claro.
- Explicar su relación con concurrencia, paralelismo o sistemas distribuidos.
- Identificar beneficios, límites y ejemplos.
- Relacionar los conceptos con Flutter, ASP.NET Core y Google Cloud.

1. Árboles balanceados

Concepto

Un árbol balanceado mantiene alturas similares entre sus subárboles para evitar recorridos muy largos. En un AVL, la diferencia de altura entre los hijos de cada nodo se limita y se corrige con rotaciones. En árboles rojo-negro se aplican reglas de color y rotaciones que garantizan una altura logarítmica.

Importancia y funcionamiento

Su búsqueda, inserción y eliminación se mantienen en O(log n). En sistemas paralelos ayudan a repartir búsquedas por ramas y a reducir puntos calientes. Una base de datos puede usar estructuras balanceadas para localizar registros e índices sin recorrer todos los datos.

Aplicación o ejemplo

En UserFlow, PostgreSQL utiliza índices para acelerar búsquedas por identificador o correo. El concepto explica por qué un índice bien organizado responde mejor que un recorrido completo de usuarios.

2. Técnica Pointer Jumping

Concepto

Pointer Jumping acelera el seguimiento de enlaces haciendo que cada elemento salte hacia el elemento apuntado por su sucesor. En cada ronda, la distancia recorrida se duplica. Una cadena de longitud n puede resolverse en aproximadamente O(log n) rondas paralelas.

Importancia y funcionamiento

Se usa en listas enlazadas, cálculo de raíces en bosques y componentes conexos. Cada procesador actualiza su puntero con información de la ronda anterior, por lo que se necesita sincronización entre etapas.

Aplicación o ejemplo

Puede imaginarse una cadena de supervisores: en una ronda cada empleado conoce a su jefe; después conoce al jefe de su jefe. Repetir el salto permite llegar rápidamente a la raíz de la jerarquía.

3. Divide y vencerás

Concepto

Divide y vencerás separa un problema en subproblemas más pequeños, resuelve cada uno y combina sus resultados. Merge Sort, Quick Sort y la búsqueda binaria son ejemplos clásicos.

Importancia y funcionamiento

Los subproblemas independientes pueden ejecutarse de manera concurrente. El rendimiento depende del costo de dividir, combinar y sincronizar. Si una rama recibe mucho más trabajo, se pierde parte de la ventaja paralela.

Aplicación o ejemplo

La carga de varios archivos puede dividirse en una tarea por archivo. Future.wait inicia esas tareas y combina los resultados cuando todas terminan.

4. Particionamiento

Concepto

Particionar significa distribuir datos o trabajo en grupos. El particionamiento puede ser por bloques, por rangos, cíclico o mediante hash.

Importancia y funcionamiento

Un buen reparto reduce comunicación y mantiene ocupados los recursos. Un reparto deficiente deja procesos ociosos mientras otros siguen trabajando. En bases de datos, particionar también puede reducir la cantidad de filas examinadas.

Aplicación o ejemplo

Cloud Run crea varias instancias cuando aumenta la demanda. Cada solicitud se asigna a una instancia disponible. Cloud SQL y el almacenamiento central conservan el estado compartido.

5. Técnica de Pipelining

Concepto

Pipelining divide un proceso en etapas y permite que diferentes elementos avancen simultáneamente por etapas distintas. Se parece a una línea de ensamblaje.

Importancia y funcionamiento

El rendimiento queda limitado por la etapa más lenta. Los buffers permiten absorber diferencias temporales, pero un exceso aumenta memoria y latencia. Los procesadores usan pipelines para solapar búsqueda, decodificación y ejecución de instrucciones.

Aplicación o ejemplo

El pipeline de GitHub Actions ejecuta restauración, pruebas, compilación, construcción Docker y despliegue. Cada fase produce una salida verificable para la siguiente.

6. Aceleramiento en cascada

Concepto

El aceleramiento en cascada combina optimizaciones en varios niveles. Una mejora prepara o amplifica la siguiente, por ejemplo algoritmo eficiente, paralelismo, caché, red y escalado de infraestructura.

Importancia y funcionamiento

Las mejoras no siempre se multiplican de forma perfecta porque aparecen límites secuenciales, comunicación y contención. La ley de Amdahl recuerda que la parte no paralelizable limita el aceleramiento total.

Aplicación o ejemplo

UserFlow combina Future.wait en el cliente, endpoints asíncronos en ASP.NET Core, índices y conexiones de PostgreSQL, contenedores y escalado de Cloud Run.

7. Técnica Symmetry Breaking

Concepto

Cuando varios procesos tienen el mismo estado y las mismas opciones, pueden intentar la misma acción a la vez. Symmetry Breaking introduce una diferencia, como un identificador, prioridad, turno, elección aleatoria o líder.

Importancia y funcionamiento

La técnica permite elegir coordinadores, asignar recursos y evitar decisiones idénticas. Es fundamental en algoritmos distribuidos y en problemas como elección de líder o coloreado de grafos.

Aplicación o ejemplo

Los identificadores únicos de usuarios, archivos, ejecuciones y recursos de Terraform permiten distinguir participantes y aplicar decisiones deterministas.

8. Cluster

Concepto

Un cluster es un conjunto de computadoras que colaboran como una plataforma. Los nodos pueden repartir cómputo, almacenar réplicas o asumir el trabajo de un nodo que falla.

Importancia y funcionamiento

Sus ventajas incluyen escalabilidad horizontal y tolerancia a fallos. Sus retos incluyen consistencia, latencia de red, observabilidad y coordinación.

Aplicación o ejemplo

Cloud Run oculta la administración del cluster: Google inicia y detiene instancias del contenedor. El desarrollador configura límites de escalado y trabaja con el servicio lógico.

9. Balanceo de cargas

Concepto

El balanceador distribuye solicitudes entre servidores saludables. Puede usar ronda circular, menor número de conexiones, pesos, ubicación o hashing.

Importancia y funcionamiento

El objetivo no es solo repartir por igual, sino reducir latencia y evitar sobrecarga. Las comprobaciones de salud impiden enviar tráfico a instancias defectuosas.

Aplicación o ejemplo

API Gateway recibe una dirección pública y Cloud Run distribuye solicitudes entre instancias. El endpoint /health permite comprobar que el backend responde.

10. Sistema operativo

Concepto

El sistema operativo administra procesadores, memoria, archivos, dispositivos y seguridad. Su planificador decide qué hilo utiliza cada núcleo y durante cuánto tiempo.

Importancia y funcionamiento

Los procesos aíslan espacios de memoria; los hilos comparten recursos del proceso. Cambiar de contexto tiene costo, por lo que crear más hilos no garantiza mayor velocidad.

Aplicación o ejemplo

El contenedor ejecuta ASP.NET Core sobre Linux administrado por Cloud Run. Android administra el proceso Flutter, la red y el acceso a archivos del teléfono.

11. Procesador de tareas

Concepto

Un procesador de tareas recibe trabajos, los coloca en una cola y asigna trabajadores. Puede reintentar, limitar concurrencia, registrar resultados y mover fallos persistentes a una cola especial.

Importancia y funcionamiento

Este patrón desacopla la recepción del trabajo de su ejecución. Resulta útil para imágenes, correos, reportes o tareas que no deben bloquear una solicitud web.

Aplicación o ejemplo
Como ampliación, UserFlow podría enviar cada archivo a una cola para análisis antivirus o generación de miniaturas, mientras la API responde que el trabajo fue aceptado.

12. Nexus

Concepto
Nexus Repository es un administrador de artefactos. Guarda paquetes, bibliotecas e imágenes de contenedor con versiones y permisos. También puede funcionar como proxy y caché de repositorios externos.

Importancia y funcionamiento
Un repositorio central mejora trazabilidad y evita depender de descargas externas en cada compilación. La política de retención evita acumular versiones innecesarias.

Aplicación o ejemplo
El proyecto usa Artifact Registry, que cumple una función equivalente para la imagen userflow-api. Cloud Run despliega la etiqueta publicada por el pipeline.

13. Repositorios de imágenes

Concepto
Un repositorio de imágenes almacena imágenes OCI/Docker por nombre y etiqueta. Cada imagen contiene capas inmutables y puede identificarse por un digest criptográfico.

Importancia y funcionamiento
El flujo normal consiste en construir, analizar, publicar y desplegar. Las etiquetas son cómodas, mientras que los digests ofrecen una referencia inmutable y más reproducible.

Aplicación o ejemplo
Cloud Build construye la imagen desde el Dockerfile, la publica en Artifact Registry y Cloud Run inicia el contenedor desde ese repositorio.

14. Pub/Sub

Concepto
Pub/Sub aplica el modelo publicar-suscribir. Un productor publica mensajes en un tema sin conocer a los consumidores. Una o más suscripciones entregan los mensajes a trabajadores independientes.

Importancia y funcionamiento
El patrón facilita sistemas desacoplados y escalables. El consumidor debe tolerar duplicados, confirmar mensajes procesados y definir reintentos o una cola de mensajes fallidos.

Aplicación o ejemplo

UserFlow podría publicar un evento ArchivoSubido. Distintos consumidores podrían registrar auditoría, generar miniaturas o notificar al usuario sin modificar el endpoint principal.

15. GitHub Actions

Concepto

GitHub Actions automatiza tareas mediante workflows YAML activados por eventos como push o pull request. Los jobs contienen pasos que se ejecutan en runners.

Importancia y funcionamiento

Los secretos evitan guardar credenciales en el repositorio. Workload Identity Federation mejora la seguridad porque entrega credenciales temporales y evita claves JSON permanentes.

Aplicación o ejemplo

El workflow de UserFlow compila y prueba .NET, autentica con GCP, construye Docker, ejecuta Terraform y despliega Cloud Run. La ejecución verde constituye evidencia reproducible de CI/CD.

Actividad práctica

Demostración propuesta: comparar la carga secuencial y concurrente de varios archivos. En el modo secuencial se espera cada respuesta antes de iniciar la siguiente. En el modo concurrente se crean todos los Future y Future.wait espera el grupo. La pantalla registra milisegundos y calcula el porcentaje de mejora. El resultado depende de red, tamaño de archivo, límites del servidor y número de conexiones; por eso debe repetirse la prueba y explicar que concurrencia no garantiza siempre una aceleración.

Parte	Qué mostrar	Qué explicar
Future.wait	Código del ViewModel	Inicia futuros independientes y espera todos los resultados.
Pipeline	Ejecución verde en Actions	Automatiza build, test, Docker, Terraform y despliegue.
Cluster/balanceo	Cloud Run	Google distribuye solicitudes y escala instancias.
Repositorio	Artifact Registry	Conserva la imagen Docker que se despliega.
Seguridad	JWT y Secret Manager	El token identifica al usuario y los secretos no están en Git.

Conclusiones

Las técnicas estudiadas resuelven problemas distintos. Divide y vencerás, particionamiento y Pointer Jumping organizan algoritmos; pipelining y aceleramiento en cascada aprovechan etapas y niveles; Symmetry Breaking coordina participantes; clusters, balanceadores, sistemas operativos y procesadores de tareas administran recursos; Nexus, repositorios de imágenes, Pub/Sub y GitHub Actions permiten construir y operar plataformas distribuidas. UserFlow integra varios de estos principios en una solución demostrable.

Referencias

1. Cormen, T. et al. Introduction to Algorithms. MIT Press.
2. Grama, A. et al. Introduction to Parallel Computing. Addison-Wesley.
3. Tanenbaum, A. Modern Operating Systems. Pearson.
4. Google Cloud. Documentación de Cloud Run, API Gateway, Cloud SQL, Pub/Sub y Artifact Registry. https://cloud.google.com/docs
5. GitHub Docs. Understanding GitHub Actions. https://docs.github.com/actions
6. Sonatype Help. Nexus Repository documentation. https://help.sonatype.com/en/sonatype-nexus-repository.html
7. Flutter API. Future.wait. https://api.dart.dev/dart-async/Future/wait.html